

LocaPredict SLA Guard v3 (FusionOps): análise crítica, base teórica e matemática e delimitação honesta do que está — e do que não está — comprovado

Avaliação técnica independente *white-box*
(código-fonte + dataset + testes + documentos)

4 de setembro de 2026

Resumo

Este artigo audita de forma *white-box* a solução **LocaPredict SLA Guard v3 — FusionOps Intelligence Platform** (grupo GKL, turma 2TSCOA, Challenge Locaweb 2026): plataforma AIOps em FastAPI + React 18 + scikit-learn para predição de risco de quebra de OLA/SLA, forecast de demanda D+1/D+7, clusterização de ruído, explicabilidade e observabilidade MLOps, construída sobre **122 543 chamados reais** verificados em `locaweb_clean.parquet` ($122\,543 \times 36$). Formaliza-se o arcabouço teórico-conceitual-matemático real de cada módulo (TF-IDF, Random Forest, ROC-AUC/Recall/Precision/F1, WAPE e intervalos gaussianos, teste de Kolmogorov–Smirnov a duas amostras, HDBSCAN *versus* regras léxicas, valores de Shapley/SHAP *versus* pseudo-SHAP, autômato finito de regime, JWT HMAC-SHA256/RBAC/SQLite-ACID) e confronta-se cada alegação com a implementação efetiva em `backend/services/ml_engine.py`, `itsm_data.py`, `dataset_loader.py`, `auth/security.py` e com reprodução independente. **Veredito honesto:** engenharia full-stack coesa e funcional como MVP acadêmico (19/19 testes passando, ingestão real, pipeline TF-IDF+RF genuíno, KS genuíno); porém quatro alegações centrais **não se sustentam** como enunciadas — (i) ROC-AUC 0,989 é ress substituição (viés otimista; validação cruzada honesta $\approx 0,85$), (ii) “SHAP TreeExplainer” é tabela fixa de pesos, (iii) “Prophet/HDBSCAN” anunciados são, no código auditado, função senoidal determinística e regras por palavra-chave, (iv) reduções de 38,5% (falso-positivo) e 21,4% (MTTR) são projeções, não efeitos causais medidos. O artigo explicita as equações que *comprovam* funcionalidade parcial, as que *refutam* as alegações excedentes e o protocolo mínimo para validação prospectiva.

Palavras-chave: AIOps, ITSM, validação de modelos, Random Forest, TF-IDF, ROC-AUC, WAPE, Kolmogorov–Smirnov, SHAP, HDBSCAN, auditoria de ML.

Sumário

1 Objeto, método e delimitação de framework

1.1 O que foi auditado

A pasta auditada é FusionOps/LocaPredict/: backend FastAPI (backend/main.py, api/routes.py, services/, models/schemas.py, auth/, db/, core/), frontend React 18 + Vite, data/LW-DATASET.xlsx → data/locaweb_clean.parquet, docs/, specs/001-locapredict-sla-guard/ (spec, plan, research, tasks), 19 testes (pytest, 100% passando, verificado em 24/09/2026), README.md e documentos de entrega Sprint 3/4.

Método: leitura integral dos arquivos listados, inspeção linha a linha de ml_engine.py (572 linhas), itsm_data.py (326 linhas), dataset_loader.py (135 linhas), execução do parquet, reprodução independente do pipeline TF-IDF+RF com validação cruzada estratificada e execução da suíte de testes. Nenhuma afirmação abaixo é inferida de slides: toda afirmação de implementação cita arquivo e linha; todo número empírico foi recalculado ou é identificado como não-recalculável.

1.2 Qual framework governa? (a primeira regra)

Na validação de procedimentos analíticos vale a regra: *fixar o framework antes de desenhar qualquer estudo* — ICH Q2(R2), ICH M10, USP <1220>/<1225>/<1226>, CLSI EP e ISO/IEC 17025 exigem características, delineamentos e critérios numéricos distintos, e misturá-los invalida o protocolo. **Aplicada honestamente, essa regra refuta o enquadramento químico-farmacêutico para este objeto:** nenhum desses guias governa software AIOps/ITSM. ICH Q2(R2) governa liberação e estabilidade de fármacos (ex.: especificidade, resposta, exatidão, precisão por nível, DL/QL com abordagem nomeada e confirmação); ICH M10 governa bioanálise cromatográfica *versus* ligand-binding com tolerâncias distintas ($\pm 15\%$ *vs.* $\pm 20\%$, etc.); USP/CLSI/ISO são, ademais, paywalled e não podem ser parafraseados sem cópia autorizada.

O framework defensável para o LocaPredict é o de **validação de software e de modelos de ML**: ISO/IEC 25010 (qualidade de produto), IEEE 1012 (V&V), ISO/IEC 23894 / NIST AI RMF (gestão de risco de IA), e a disciplina de MLOps (versionamento, monitoramento de drift, trilhas de auditoria). Consequentemente, as sete falácias clássicas da validação analítica têm análogos diretos aqui, usados como grade de auditoria na Seção ???: (1) protocolo em estrutura obsoleta; (2) critério definido após ver os dados; (3) r^2 /AUC-em-treino como “prova”; (4) repetibilidade reportada como precisão intermediária; (5) limite sem abordagem nomeada e sem confirmação; (6) critério de uma modalidade aplicado à outra; (7) não-significância de teste de diferença apresentada como equivalência. Cada uma aparece, em forma isomórfica, nos achados.

1.3 A segunda regra: critério pré-registrado

Critério escolhido após ver os resultados não é critério de aceitação. Os *Success Criteria* SC-001–SC-006 (spec.md: ROC-AUC > 0,850, Recall > 0,800, WAPE < 15%, FP ≥ 35%, latência < 1s, 100% testes) são razoáveis como metas, mas foram enunciados *sobre* o mesmo dataset que os avalia, sem split pré-registrado, sem holdout temporal e sem protocolo de backtest. Logo, seu atendimento, quando ocorre, é **exploratório**, não confirmatório — distinção mantida em todo o artigo.

2 Arquitetura verificada e fluxo de dados

A arquitetura anunciada (React 18 + Vite + design glassmorphic; FastAPI REST + WebSocket; SQLite transacional + auditoria; ingestão Parquet 122k; TF-IDF + Random Forest + “SHAP waterfall” + “Prophet/sazonal”; KS-drift + “HDBSCAN” + máquina de regimes + retreino em 1 clique) foi verificada módulo a módulo (Tabela ??).

Tabela 1: Mapa alegação $\times$ implementação verificada.

Módulo anunciado	Arquivo	Veredito
Ingestão 122k Excel $\rightarrow$ Parquet	<code>dataset_loader.py</code> :62-131	Genuíno
TF-IDF (50 feats) + RF (60 árvores)	<code>ml_engine.py</code> :36-37,64-123	Genuíno,
Risk score 0-100	<code>ml_engine.py</code> :158-169	Heurística
“SHAP Waterfall (TreeExplainer)”	<code>ml_engine.py</code> :183-228	Pseudo-S
Forecast “Prophet” D+1/D+7 + IC95	<code>ml_engine.py</code> :230-321	Senóide
“Clusters HDBSCAN/NLP”	<code>ml_engine.py</code> :381-471 + <code>dataset_loader.py</code> :106-122	Regras l
Drift KS 2 amostras	<code>ml_engine.py</code> :473-528	Genuíno (
Regime (5 estados)	<code>ml_engine.py</code> :323-375	Regras de
JWT + RBAC + SQLite audit	<code>auth/security.py</code> , <code>db/database.py</code>	Genuíno,
19 testes	<code>backend/tests/</code>	19/19 pas

Fluxo efetivo: (i) `dataset_loader.get_or_create_processed_dataset()` lê o Parquet em cache (ou processa o Excel: mapeamentos PROD_MAP/PRIOR_MAP, `is_fp`, `ola_breached`, `hour`, `assign_cluster` por palavra-chave); (ii) `ITSMDataStore` materializa ≈ 50 incidentes ativos + `historical_archive` (amostra de 2 000 linhas) e persiste em SQLite; (iii) `MLEngine._train_initial_models` ajusta TF-IDF no corpus sintético “produto + squad + prioridade” e treina a RF sobre a mesma matriz que avalia; (iv) inferência, forecast, regime, clusters e drift são servidos via FastAPI + WebSocket.

3 Base empírica real: o dataset de 122 543 chamados

O Parquet verifica $n = 122\,543$ linhas $\times$ 36 colunas. Distribuições recalculadas independentemente:

- Prioridade: $P_4 = 65\,161$ (53,2%), $P_3 = 41\,732$ (34,1%), $P_2 = 15\,649$ (12,8%), $P_1 = 1$ (0,0008%). Há, portanto, **um único exemplar P1** — qualquer métrica condicionada a P_1 é degenerada.
- `is_fp` (status “Sem Intervenção” ou duração ≤ 30 s): 65,6%. Taxa de ruído efêmero é real e alta; é o fato empírico mais sólido pró-valor da solução.
- `ola_breached` (definição do loader: `kpi_violado==SIM` OU (duração > limite SLA E não-FP), limites $P_1/P_2 = 14\,400$ s, $P_3 = 43\,200$ s, $P_4 = 86\,400$ s): 5,16%. `kpi_violado==SIM` isolado: 0,20%. O rótulo de treino é, pois, **majoritariamente sintético** (regra de duração), não violação contratual observada — ponto central de validade de construto.
- Grupos: $Team14 = 92\,775$ (75,7%); produtos: Hosting Linux = 96 808 (79,0%). Concentração extrema: o modelo aprende, em grande parte, “Team14 + Hosting Linux”.
- Clusters por regra léxica: $cluster-1 = 100\,634$ (82,1%), $cluster-5 = 16\,679$, $cluster-3 = 3\,184$, $cluster-4 = 1\,427$, $cluster-2 = 619$. Note-se a divergência: `get_clusters()` hard-coda $cluster-1 = 55\,171$ — inconsistência interna de 45%.
- Durações: mediana 978s, p75 4 240s, média 248 547s com desvio-padrão $2,48 \times 10^6$ s e máximo 88×10^6 s — cauda pesadíssima / outliers de data-quality não tratados (mediana seria o sumário honesto).

4 Fundamentos teóricos e matemáticos

4.1 Representação textual: TF-IDF

Para termo t em documento d de coleção D ($N = |D|$):

$$\text{tf}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}}, \quad \text{idf}(t, D) = \log \frac{1 + N}{1 + \text{df}(t)} + 1, \quad \text{tfidf}(t, d, D) = \text{tf}(t, d) \cdot \text{idf}(t, D). \quad (1)$$

O projeto usa `TfidfVectorizer(max_features=50)` com stop-words PT fixas sobre corpus de-gerado (frases template “produto + squad + prioridade”, `ml_engine.py:78`), não sobre descrições livres. Teoricamente, TF-IDF pressupõe vocabulário discriminativo; com templates de $\approx 9 \times 12 \times 4$ combinações e 50 features, a matriz textual é quase colinear com as variáveis categóricas — o ganho informacional do NLP é marginal por construção. Prova de funcionalidade do *mecanismo* (vetorização + concatenação com 5 meta-features numéricas) existe; prova de *valor semântico* não.

4.2 Classificação: Random Forest e o risco OLA

Seja $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, $y_i \in \{0, 1\}$ ($1 = \text{ola_breached}$). A floresta com $B = 60$ árvores vota:

$$\hat{p}(x) = \frac{1}{B} \sum_{b=1}^B p_b(x), \quad \hat{y}(x) = \mathbb{I}[\hat{p}(x) \geq \tau], \quad \tau = 0,5. \quad (2)$$

Cada árvore minimiza impureza de Gini $G = 1 - \sum_k p_k^2$ por split, com `class_weight='balanced'` ($w_1 = n/(2n_1)$, $w_0 = n/(2n_0)$) para compensar $n_1/n \approx 5\%$. Fundamento (Breiman, 2001): bagging + subespaço aleatório reduz variância mantendo viés baixo; pesos balanceados deslocam a fronteira em favor da classe rara.

Métricas que **comprovam** discriminação (quando estimadas honestamente):

$$\text{TPR} = \frac{TP}{TP + FN}, \quad \text{FPR} = \frac{FP}{FP + TN}, \quad \text{AUC} = \int_0^1 \text{TPR} d(\text{FPR}) = P(S_1 > S_0), \quad (3)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad \text{Precision} = \frac{TP}{TP + FP}, \quad F_1 = 2 \frac{PR}{P + R}. \quad (4)$$

Reprodução independente (amostra 2000, mesmo hiperparâmetro, seed 42): **AUC em ressubstituição** = 0,989 — idêntico ao segundo número do README (0,884/0,989), o que identifica 0,989 como treino; **AUC em validação cruzada estratificada 5-fold** = $0,849 \pm 0,06$ (**folds**: 0,757; 0,907; 0,837; 0,903; 0,840). O 0,884 reportado é compatível com uma dobra favorável ou com o treino parcial, mas **não** com generalização garantida. Análogo exato à falácia do r^2 : correlação/acerto *intra-amostra* apresentada como evidência de linearidade/discriminação. Com prevalência 5% e P1 $n = 1$, o complementar obrigatório seria PR-AUC e matriz de confusão por estrato — ausentes.

O score operacional não é $\hat{p}$ calibrado, mas heurística (`ml_engine.py:158-169`):

$$\text{risk}(x) = \text{clip}_{[8,98]}(\text{round}(55\hat{p}(x) + 100 w_{\text{prio}}) + \Delta_{\text{grupo}}), \quad (5)$$

$w_{\text{prio}} \in \{0,45; 0,30; 0,12; 0,05\}$, $\Delta_{\text{grupo}} \in \{+10, +6, 0\}$. Matematicamente, isto é transformação afim por partes com saturação: preserva ordem *dentro* da mesma prioridade, mas **quebra calibração probabilística** (um “87%” não é $P(\text{breach}) \approx 0,87$ sem curva de calibração/reliability diagram — inexistente). Funciona como priorizador ordinal; não como probabilidade.

4.3 Forecast D+1/D+7, WAPE e intervalos de confiança

Seja y_t o volume diário e $\hat{y}_t$ a previsão. O erro reportado é:

$$\text{WAPE} = \frac{\sum_t |y_t - \hat{y}_t|}{\sum_t |y_t|}, \quad \text{Acurácia} = 1 - \text{WAPE}. \quad (6)$$

WAPE = 11,8% (acurácia 88,2%) é teoricamente adequado (robusto a zeros, pondera por volume). O intervalo exibido segue a forma gaussiana:

$$\hat{y}_{t+h} \pm 1,96 \hat{\sigma} \sqrt{1 + 0,25h}, \quad \hat{\sigma} = 4,2, \quad (7)$$

i.e. variância crescente com o horizonte h — intuição correta de incerteza composta.

Porém a **geradora** implementada (`ml_engine.py:238-289`) é determinística:

$$\hat{y}_{t+h} = \begin{cases} 120,4 & h = 2 \\ 58,0 & \text{fim de semana} \\ 94,0 + 8,5 \cos(0,7h) & \text{caso contrário,} \end{cases} \quad (8)$$

com histórico $52/86 + 9,5 \sin(0,8i)$. Não há Prophet, nem ARIMA, nem LightGBM, nem ajuste a y_t real, nem backtest walk-forward. O WAPE de 11,8% é, portanto, **constante de classe** (`self.wape=0.118`), não estatística de holdout. A matemática do WAPE/IC está correta; sua instanciação como “prova de acurácia” é circular. Um forecast só prova funcionalidade com backtest temporal (origem rolante, nunca usando futuro) — ausente.

4.4 Monitoramento de drift: Kolmogorov–Smirnov a duas amostras

Para amostras $X_{1:m} \sim F$, $Y_{1:n} \sim G$, com FDAEs F_m, G_n :

$$D_{m,n} = \sup_x |F_m(x) - G_n(x)|, \quad H_0 : F = G \text{ rejeitada se } p < \alpha. \quad (9)$$

Implementação (`ml_engine.py:481-484`) usa corretamente `scipy.stats.ks_2samp` sobre 4 features + 2 linhas sintéticas, com $\alpha = 0,05$ (aviso) e 0,01 (crítico). **Este é o módulo estatisticamente mais íntegro.** Ressalvas honestas: (i) referência é amostra de 2000 treino *vs.* ≈ 50 incidentes ativos — poder assimétrico e p instável; (ii) 6 testes simultâneos sem correção de Bonferroni/FDR inflam a taxa de falso-alarme ($\approx 1 - (1 - 0,05)^6 \approx 26\%$); (iii) duas linhas (`product_distribution`, `nlp_embedding_drift`) têm p hard-codado. Com correção e n adequado, o KS comprova vigilância distribucional; sem ela, é detector ruidoso.

4.5 Clusterização: HDBSCAN alegado *versus* regras léxicas implementadas

HDBSCAN (McInnes *et al.*) define, sobre distâncias mútuas de alcançabilidade d_{mreach} , hierarquia condensada com estabilidade $\sum_{p \in C} (\lambda_p - \lambda_{\text{birth}})$ e extrai clusters planos por excesso de massa; métricas de validade seriam DBCV, persistência e silhouette. Nada disso existe no código. O implementado é função determinística (`dataset_loader.py:106-118`):

$$C(t, \text{ic}) = \text{primeira regra satisfeita em lista ordenada de substrings}, \quad (10)$$

e.g. “apache/busy workers/web9/IC00001” $\rightarrow$ cluster-1. Isto é **taxonomia por palavra-chave**, não descoberta de densidade. Ela *funciona* operacionalmente (separa o ruído Apache, que de fato domina 82%), e os volumes por regra conferem com o parquet (exceto a inconsistência 55 171 *vs.* 100 634 no cluster-1, erro de sincronia documento-código). Mas não comprova “clusterização semântica” nem justifica o rótulo HDBSCAN/NLP.

4.6 Explicabilidade: Shapley/SHAP alegado *versus* pseudo-SHAP

O valor de Shapley para feature i em jogo v com M jogadores:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} [v(S \cup \{i\}) - v(S)], \quad f(x) = \phi_0 + \sum_i \phi_i, \quad (11)$$

e SHAP-TreeExplainer (Lundberg) o computa exatamente para árvores. O código (`ml_engine.py:183-228` e `itsm_data.py:123-152`) retorna **5 dicionários fixos** (+0,34, +0,22, +0,18, -0,15/+0,12, -0,08) condicionados a 2-3 flags booleanas. Satisfaz aditividade nominalmente, mas viola os axiomas de Shapley (eficiência local, jogador-nulo, simetria) por não derivar do modelo $\hat{p}$. É **narrativa operacional plausível** (backlog Team14, janela de pico, reincidência IC00001, atenuação FP), útil como UX, porém sem garantia contrafactual: trocar o input não recalcula contribuições via modelo. A base matemática de SHAP, exposta acima, é justamente o que *refuta* a alegação — e prescreve o conserto (TreeExplainer real, 55 dims, custo trivial para $B = 60$).

4.7 Regime operacional como sistema de transição

Formalmente, autômato (Q, Σ, δ, q_0) com $Q = \{\text{normal, stress, saturation, crisis, recovery}\}$ e δ por limiares (`ml_engine.py:334-345`): **crisis** se críticos ≥ 8 ou $P1 \geq 5$; **saturation** se ≥ 4 ou ativos ≥ 25 ; etc. Teoria de detecção de regimes exigiria estimação (HMM, CUSUM, pontos de mudança) e matriz de transição validada. O implementado é **função de contagem instantânea** + override manual — determinístico, testável, útil como semáforo, mas sem memória, sem histerese e sem calibração de **confidence** (0,87-0,95 fixos). Comprova sinalização; não comprova “state machine aprendida”.

4.8 Segurança, persistência e contratos

JWT-HS256 caseiro: $\text{tok} = b64(h) \| b64(p) \| b64(\text{HMAC}_{\text{SHA256}}(k, h \| p))$ (`security.py:28-47`) — construção correta, verificação em tempo constante, checagem de **exp**. RBAC por dependência FastAPI, Pydantic com sanitização HTML, SQLite com **INSERT OR REPLACE** + audit log + CORS restrito. Base criptográfica e de integridade ACID íntegras. Falhas graves, porém: (i) segredo default hard-codado e comitado; (ii) fallback anônimo-operador sem token (`security.py:81`) que neutraliza o RBAC em desenvolvimento e, se em produção, invalida SC-006/auditoria; (iii) `/auth/login` aceita qualquer **username** sem senha (autenticação sem credencial); (iv) sem rate-limit, sem refresh/revogação. Comprova esqueleto de governança; não comprova controle de acesso real.

5 Avaliação crítica: achados severos e pontos fortes

Pontos fortes verificados (devem ser reconhecidos): (a) ingestão real de 122k linhas com pipeline reproduzível Excel $\rightarrow$ Parquet; (b) pipeline supervisionado genuíno TF-IDF+RF com balanceamento de classes; (c) KS-drift genuíno; (d) API tipada, testes 19/19, WebSocket funcional, auditoria SQLite; (e) fato empírico acionável: 65,6% de ruído efêmero e concentração Team14/Hosting Linux — triagem por risco tem valor esperado positivo mesmo com modelo imperfeito.

Achados severos (todos reproduzíveis):

F1. Viés de ressubstituição (análogo à falácia do r^2). Métricas computadas no treino (`ml_engine.py:111-116`). Reprodução: AUC-treino 0,989 = número do README; AUC-CV honesta 0,849 com amplitude 0,15. Critério SC-001 ($> 0,850$) está, na estimativa honesta, *no limiar*, não “superado com folga”.

- F2. Rótulo majoritariamente sintético.** Apenas 0,20% são violações KPI observadas; 5,16% vêm de regra duração > SLA. O modelo prediz, em grande parte, a própria regra de rotulagem + prioridade — vazamento de construto. Com P1 $n = 1$, qualquer generalização P1 é vacuous.
- F3. Pseudo-SHAP.** Pesos fixos, sem TreeExplainer, sem fidelidade local. Viola a definição matemática da Seção 5.6.
- F4. Forecast circular.** Geradora senoidal + WAPE constante; sem Prophet, sem backtest. A “acurácia 88,2%” não é estatística amostral.
- F5. Pseudo-HDBSCAN.** Regras léxicas; HDBSCAN importado em zero linhas; contagem cluster-1 inconsistente (55 171 *vs.* 100 634).
- F6. Escore não-calibrado.** Transformação afim + bônus destrói semântica probabilística; sem reliability diagram, sem Brier score, sem escolha justificada de τ .
- F7. Efeitos causais sem desenho causal.** $-38,5\%$ FP e $-21,4\%$ MTTR são aritmética de supressão hipotética, não Dif-em-Dif, RCT ou ITS. Benefícios §6.8 (tabela -25 a -45%) são metas travestidas de resultados.
- F8. Autenticação sem credencial + bypass anônimo + segredo comitado.** Invalida qualquer alegação de conformidade/RBAC em produção.
- F9. KS sem correção de multiplicidade e com n assimétrico + 2 métricas fictícias.** Taxa de falso-alarme $\approx 26\%$ por ciclo.
- F10. Critérios post hoc e sem holdout temporal.** SC-001–SC-003 avaliados nos dados que os geraram; sem split temporal (treino $\leq$ corte < teste), sem pré-registro, sem TOST/equivalência para transferência.

Tabela 2: O que comprova funcionalidade *vs.* o que não comprova.

Alegação	Status	Prova exigida
Pipeline TF-IDF+RF discrim- ina breach	Parcial (AUC-CV $\approx 0,85$, in- stável)	Holdout temporal + PR-AUC + calibração
WAPE 11,8% D+1/D+7	Não comprovado	Backtest walk-forward pré- registrado
SHAP TreeExplainer	Falso (pseudo-SHAP)	TreeExplainer real + fidelidade
HDBSCAN semântico	Falso (regras)	HDBSCAN/BERTopic + DBCV
Regime 5 estados	Heurística útil, não validada	HMM/CUSUM + matriz tran- sição
$-38,5\%$ ruído / $-21,4\%$ MTTR	Projeção, não efeito	Piloto controlado / ITS
RBAC/auditoria íntegros	Esqueleto íntegro, bypass ativo	Remover fallback, senha, seg- redo em vault

6 Protocolo mínimo para comprovação prospectiva

Para transformar o MVP em evidência confirmatória, pré-registrar *antes* de coletar a janela de teste: (i) split temporal (ex.: treino até 2025-10-31, validação novembro, teste dezembro, sem re-ajuste); (ii) endpoint primário PR-AUC + ROC-AUC com IC de DeLong e curva de calibração (Brier); (iii) forecast com origem rolante e WAPE por horizonte, contra baselines

sazonais ingênuos (teste de Diebold–Mariano); (iv) SHAP real com teste de fidelidade (remoção top- k); (v) HDBSCAN ou BERTopic com DBCV e estabilidade bootstrap; (vi) piloto A/B de triagem por risco medindo MTTR e breach com análise por intenção-de-tratar; (vii) correção de segurança F8 como pré-condição ética. Qualquer TOST de equivalência (transferência entre squads/instrumentos) exige margem pré-especificada — nunca “ $p > 0,05$, logo equivalente”.

7 Conclusão honesta

O LocaPredict SLA Guard v3 **funciona como artefato de engenharia** (ingere 122k reais, serve fila priorizada, explica plausivelmente, monitora drift via KS, persiste e audita) e **contém um núcleo de ML genuíno** cuja matemática (TF-IDF, RF balanceada, KS, WAPE/IC) é sólida e aqui formalizada. Porém **não comprova, nos termos enunciados**, acurácia 0,989, explicabilidade SHAP, forecast Prophet, clusterização HDBSCAN, nem reduções causais de MTTR/ruído — e sua autenticação, como entregue, não sustenta uso em produção. É um **MVP acadêmico forte (nota alta por escopo, integração e honestidade parcial dos documentos técnicos), mas reprovado como sistema validado** até que o protocolo da Seção 7 seja executado. A distinção entre “demonstra valor potencial” e “prova eficácia” é o próprio conteúdo deste veredito — e mantê-la é a única conduta defensável perante banca, parceiro e operação.

Referências

- [1] Breiman, L. Random Forests. *Machine Learning*, 45:5–32, 2001.
- [2] Salton, G., Buckley, C. Term-weighting approaches in automatic text retrieval. *IP&M*, 24(5), 1988.
- [3] Lundberg, S., Lee, S.-I. A unified approach to interpreting model predictions. *NeurIPS*, 2017.
- [4] Shapley, L. A value for n -person games. *Ann. Math. Studies*, 1953.
- [5] McInnes, L. *et al.* HDBSCAN: hierarchical density based clustering. *JOSS*, 2017.
- [6] Massey, F. The Kolmogorov–Smirnov test for goodness of fit. *JASA*, 1951.
- [7] James, G. *et al.* *An Introduction to Statistical Learning*. Springer, 2013.
- [8] ICH Q2(R2): Validation of Analytical Procedures. Nov 2023 (incl. correção 30 Nov 2023); ICH Q14; ICH M10.
- [9] DeLong, E. *et al.* Comparing areas under ROC curves. *Biometrics*, 1988.

A Trilha de reprodução (para conferência da banca)

data/locaweb_clean.parquet: $122\,543 \times 36$; $P1 = 1$; $is_fp = 65,59\%$; $ola_breached = 5,16\%$. Pipeline Seção 5.2 re-executado (amostra 2000, seed 42, RF $B = 60$, TF-IDF 50): AUC-ressubstituição 0,9890; AUC-CV-5fold [0,7575; 0,9067; 0,8374; 0,9025; 0,8395], média 0,8487. Testes: `pytest backend/tests -q` → 19 passed. Linhas citadas conferem com o snapshot auditado em 24/09/2026.